

„Matematyczne” opracowanie objawów wad wzroku

Lp.	Objaw	Wzór
1.	Przebłyśki, błyski światła	„Rozlewanie się” jasnych pikseli na piksele sąsiadujące. - bright-pass – należy obliczyć luminancję obrazu i stworzyć maskę wycinając tylko jasne miejsca; - konwolucja (jw.) maski z jądrem wykładniczym: $K(x, y) = e^{-k(x^2 + y^2)}$ gdzie: k – współczynnik tłumienia [Unity] Można użyć - overlay addytywny (Bloom)
2.	„Mroczki” lub „zastona” przed okiem	Rozmieszczenie półprzezroczystych obiektów o różnych kształtach, poruszających się. - Kształty mroczków: - Plamki (perlin noise) $m_{spot}(x, y) = a \bullet \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right)$ - Pierścienie (DoG ring) $m_{ring}(x, y) = a \bullet \max\left(0, e^{\frac{-r^2}{2\sigma_1^2}} - b e^{\frac{-r^2}{2\sigma_2^2}}\right)$ - Włókna $m_{ellip}(x, y) = a \bullet \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right)$ $\begin{aligned} x' &= (x - x_0)\cos\theta + (y - y_0)\sin\theta \\ y' &= (x - x_0)\sin\theta + (y - y_0)\cos\theta \end{aligned}$ $\sigma_x \gg \sigma_y$ gdzie: x_0, y_0 – centrum a – amplituda σ – promień rozmycia - Maska mroczków: $M(x, y) = \sum_i m_i(x, y)$ 2. Ruch + opóźnienie:

Lp.	Objaw	Wzór
		$s(t + 1) = s(t) - c \Delta e(t) + \epsilon$ gdzie: • S – pozycja mroczków (x, y) • C – siła opóźnienia • Δe – wektor przemieszczenia gałki ocznej • ϵ - dryf 3. Mieszanie z obrazem: $G(x, y) = I(x, y)[1 - (M * K)(x, y)]$ gdzie: • G - obraz po rozmyciu, • I – obraz przed rozmyciem, • K - kernel, jądro Gaussa [Unity] • maska plan - narysowanie kilku kształtów i przesunięcie ich w czasie • texture2D • Gotowy shader: https://zenodo.org/records/7272549
3.	Zamglenia, zamglone widzenie	Model atmosferyczny/ Koschmiedera Połączenie efektu rozbłysku ze zmniejszeniem kontrastu. 1. Przebłyski światła – jw. 2. Zmniejszenie kontrastu $G(x, y) = I(x, y) \cdot e^{-\beta} + A \cdot (1 - e^{-\beta})$ gdzie: • A – kolor „mgły” (biały/szary) • β – siła zamglenia [Unity] • post-process shader na podstawie odległości piksela; • Bloom efekt rozproszenia światła na siatkówce • Small Gaussian blur symuluje utratę ostrości przez film łzowy, • Color Grading - obniża kontrast i saturację
4.	Męty, „latające” plamki, kropki, nitki, pajęczyny	Jak w punkcie 2. /// Konwolucja (p. wrażenie rozmycia obrazu) maski dla każdego piksela obrazu. Lub $O(x, y) = I(x, y) \cdot M(x, y)$ $O_{x, y} = I_{x, y} \cdot M_{x, y}$ Gdzie: • I - oryginalny obraz

Lp.	Objaw	Wzór
		• M - maska opisująca poziom zaburzenia [0,1] • O - obraz wynikowy [Unity] • Overlay z teksturą/maską (Texture2D z ciemnymi elementami na przezroczystym tle) Albo tak jak koledzy na inżynierce zrobili - narysowali maskę z czarnymi kropkami/kształtami LUB pobawić się animację takich kształtów, gdzie zmienną będzie intensywność takich plam i ich "przezroczystość/zaciemnienie" Może przydatne: https://www.youtube.com/watch?v=24BzdRntHfo [Unity] • dodanie narysowanych masek z mętami • Gaussian blur - rozmyte cienie • bloom - przyciemnienie okolic jasnych plam https://www.cg.tuwien.ac.at/research/publications/2020/KROESL-2020-SVI/KROESL-2020-SVI-thesis.pdf
5.	Stopniowe zmniejszanie pola widzenia	Radialna winieta progresywna $r = \sqrt{x^2 + y^2}, \quad \omega(r) = \text{smoothstep}(r_0, r_1, r), \quad O = I \cdot (1 - \omega(r))$ gdzie: • O - obraz wynikowy • I - obraz źródłowy • r - odległość piksela od środka pola widzenia • w(r) - winieta • r₀ - promień od którego zaczyna się przyciemnienie • r₁ - promień, przy którym obraz jest całkowicie czarny https://www.cg.tuwien.ac.at/research/publications/2020/KROESL-2020-SVI/KROESL-2020-SVI-thesis.pdf [Unity] • post-processing Vignette
6.	Pogorszenie ostrości wzroku	Filtr dolnoprzepustowy - przy pogorszeniu ostrości wzroku następuje zanik wysokich częstotliwości $\hat{g}(\mathbf{f}) = \hat{f}(\mathbf{f}) \cdot H(\mathbf{f})$

Lp.	Objaw	Wzór
		$H(\mathbf{f}) = \exp\left(-\frac{\ \mathbf{f}\ ^2}{f_c^2}\right)$ gdzie: • $f(x, y)$ - obraz bazowy • $g(x, y)$ - obraz wynikowy • $\hat{f}(\mathbf{f})$ - transformata Fouriera obrazu wejściowego, opisująca jego składniki częstotliwościowe. • $g^{\wedge}(\mathbf{f})$ - transformata obrazu po filtracji. • $\mathbf{f}$ - wektor częstotliwości przestrzennych (f_x, f_y). • $H(\mathbf{f})$ - funkcja przenoszenia filtra (ang. <i>transfer function</i>). • f_c (f_c) - częstotliwość odcięcia, określająca jak bardzo obraz jest rozmyty; mniejsze $f_c \rightarrow$ większe rozmycie. • $\ \mathbf{f}\ ^2 = f_x^2 + f_y^2$ - długość wektora częstotliwości (czyli „jak bardzo szczegółowy” jest element obrazu). https://www.cg.tuwien.ac.at/research/publications/2020/KROESL-2020-SVI/KROESL-2020-SVI-thesis.pdf [Unity] • Bloom effect (light bloom, glow),
7.	Rozmazany, wykrzywiony obraz	Model kwadratowy $p_d = (p - c)(1 - kr^2) + c$ gdzie: • $p = (x, y)$ - współrzędne piksela • p_d - współrzędne po zniekształceniu • $r = \sqrt{x^2 + y^2}$ - odległość od środka obrazu • k, k_1, k_2 - współczynniki dystorsji • c - środek zniekształcenia https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html [Unity] • post-processing shader z efektem dystorsji https://learn.unity.com/tutorial/post-processing-effects-lens-distortion
8.	Zniekształcenia linii prostych	Pole przemieszczeń $g(x, y) = f(x + \delta_x(x, y), y + \delta_y(x, y))$ gdzie: • $\delta(x, y) = (\delta_x, \delta_y)$ - gładkie pole wektorowe przemieszczenia • $f(x, y)$ - obraz bazowy • $g(x, y)$ - obraz wynikowy • $\delta_x(x, y), \delta_y(x, y)$ - funkcje przesunięcia

Lp.	Objaw	Wzór
		https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf [Unity] • Mapa przemieszczeń • shader offset UV
9.	Ciemna plama w centrum pola widzenia	Gaussowska maska centralna (podobnie jak w przykładzie 2.) $O(x, y) = I(x, y) \cdot (1 - M_{Gauss}(x, y))$ gdzie maska ma postać: $M_{Gauss}(x, y) = \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right)$ gdzie: • $I(x, y)$ - obraz bazowy • $O(x, y)$ - obraz wynikowy • $M(x, y)$ - maska gaussowska • σ - promień rozmycia maski https://www.researchgate.net/publication/228951905_Visual_eye_disease_simulator https://www.cg.tuwien.ac.at/research/publications/2020/KROESL-2020-SVI/KROESL-2020-SVI-thesis.pdf Unity • maska plan - rysujesz kilka kropek/ kształtów i przesuwasz je w czasie • texture2D
10.	Problemy z rozpoznawaniem kolorów	Proste macierze 3x3 $\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = M_{type} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$ gdzie: • M_{type} - macierz danego typu wady (protan/deutan/tritan) • R, G, B - składowe koloru wejściowego • R', G', B' - składowe po symulacji https://colour.readthedocs.io/en/v0.3.16_b/colour.blindness.html [Unity] • Color Adjustments / Color Grading (Post-Processing) - ręczna korekcja barw, nasycenia i balansu kolorów • Custom Color Matrix Shader - przemnożenie koloru piksela przez macierz 3x3

Lp.	Objaw	Wzór
11.	Zwężenie pola widzenia (widzenie lunetowe)	wykorzystanie apertury kołowej - widzenie przez tunel $O(x, y) = I(x, y)1_{r \leq R} + I(x, y)(1 - \omega)1_{r > R}$ gdzie: • $1_{r \leq R}$ - obszar widzenia • $1_{r > R}$ - obszar przyciemniony • ω - współczynnik zaciemnienia (od 0 do 1), • R - promień pola widzenia https://www.cg.tuwien.ac.at/research/publications/2020/KROESL-2020-SVI/KROESL-2020-SVI-thesis.pdf [Unity] • Vignette (Post-Processing) - efekt tunelu • Custom Mask Shader - twarda maska kołowa
12.	Spadek wyrazistości kolorów	Desaturacja - przejście od oryginalnego koloru c do odcienia szarości o tej samej luminancji Y $Y = \omega_R R + \omega_G G + \omega_B B$ $c' = Y + s(c - Y), \quad 0 \leq s \leq 1$ gdzie: • w_R, w_G, w_B - wagi luminancji • Y - luminancja • s - współczynnik nasycenia (1 - pełny kolor, 0 - całkowicie szary) • c' - kolor po desaturacji https://www.cg.tuwien.ac.at/research/publications/2020/KROESL-2020-SVI/KROESL-2020-SVI-thesis.pdf [Unity] • post-processing color grading - zmniejszenie parametrów saturation i kontrast
13.	Podwójne widzenie	Jeśli mamy obraz $I(x, y)$, a obiekt zajmuje obszar R, to symulowane podwójne widzenie $I'(x, y)$ można opisać jako (sumowanie przesuniętych obrazów): $I'(x, y) = I(x, y) + \alpha \cdot I(x - \Delta x, y - \Delta y)$ gdzie: • $I(x, y)$ - obraz bazowy • $I'(x, y)$ - obraz wynikowy • $\Delta x, \Delta y$ - przesunięcie drugiego obrazu [piksele] • α - współczynnik przezroczystości drugiego obrazu (0-1), aby obraz nie był całkowicie nałożony https://www.researchgate.net/publication/228951905_Visual_eye_disease_simul

Lp.	Objaw	Wzór
		ator Efekt: jeden obiekt jest widoczny jako dwa, lekko przesunięte względem siebie [Unity] • grabPass - tworzenie kopii ekranu + przesunięcie go o Δx i Δy z przezroczystością α • Należy stworzyć drugi przesunięty obraz sceny (o kilka pikseli) • Zastosować częściową przezroczystość
14.	Problemy z widzeniem w nocy	Mnożenie i dodawanie ze stałą: $g(x) = \alpha f(x) + \beta$ $g_x = \alpha f_x + \beta$ Gdzie: • $\alpha > 0$ - wzmocnienie • β - przesunięcie • $f(x)$ - piksele obrazu źródłowego • $g(x)$ - piksele obrazu wynikowego Inaczej (wygodniej): $g(i, j) = \alpha \cdot f(i, j) + \beta$ $g_{i, j} = \alpha \cdot f_{i, j} + \beta$ Gdzie: • i, j - współrzędne piksela (i-ty wiersz, j-ta kolumna) https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html [Unity] Można użyć: • Ciemne brzegi - Vignette (Post-Processing), • Ziarno i zakłócenia - Film Grain, z mocniejszym szumem w ciemnych miejscach, • Adaptacja oka do ciemności - Auto Exposure/Eye Adaptation, • Desaturacja i brak kontrastu - Color Grading (obniżona saturacja, kontrast, przesunięcie kolorów w stronę niebieskiego i zielonego), • Rozmycie i nieostre widzenie - Blur lub Chromatic Aberration, Depth of Field. Asset store: Night Vision Effect https://www.sharpcoderblog.com/blog/night-vision-image-effect-in-unity-3d
15.	Widzenie aureoli wokół światła	Rozmycie Gaussa wokół źródła światła/konwolucja obrazu z jądrem gaussowskim o większym promieniu (p. wrażenie rozmycia obrazu) Rozmycie Gaussa (w dwóch wymiarach):

Lp.	Objaw	Wzór
		$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ Gdzie: • x - odległość od środka na osi poziomej • y - odległość od środka na osi pionowej • G - odchylenie standardowe rozkładu Gaussa (promień rozmycia) • σ - promień rozmycia • Centrum jest w (0,0) https://en.wikipedia.org/wiki/Gaussian_blur [Unity] Bloom effect (light bloom, glow): • Używany w post-processing-u • zastosowanie filtra Gaussowskiego lub aproksymacji rozmycia https://en.wikipedia.org/wiki/Bloom_(shader_effect) https://developer.playcanvas.com/user-manual/graphics/posteffects/bloom/ Shader Halo (Color, Size): https://docs.unity3d.com/530/Documentation/Manual/class-Halo.html
16.	Uczucie drżenia obrazu	Proste, losowe drżenie (kamery): $C(t) = C_0 + \Delta C(t)$ gdzie: $\Delta C(t) = A\eta(t)e^{-kt}$ • C_0 - pozycja spoczynkowa kamery • A - maksymalna amplituda drżenia • $\eta(t)$ - f. losowa np. szum Perlina generująca wektor $[x(t), y(t), z(t)]$ • k - współczynnik tłumienia [Unity] Camera shake - darmowy aset https://github.com/andersonaddo/EZ-Camera-Shake-Unity Losowa zmiana pozycji kamery oraz wykorzystanie Animation Curve https://www.youtube.com/watch?v=BQGTdRhGmE4
17.	Zawroty głowy, uczucie chwiania	Sinusoidalna rotacja kamery: $\theta(t) = A \cdot \sin(2\pi ft + \phi)$

Lp.	Objaw	Wzór
		gdzie: • A - amplituda obrotu • f - częstotliwość kołysania • t - czas • Φ - faza początkowa Efekt chwiania się (dodanie przesunięć liniowych) $x(t) = X_0 + A_x \cdot \sin(2\pi f_x t + \phi_x)$ $y(t) = y_0 + A_y \cdot \sin(2\pi f_y t + \phi_y)$ [Unity] Filtr Simple Radial Blur: https://www.shadertoy.com/view/XsfSDs Do zastosowania: • Drgania kamery - losowe przesunięcia pozycji kamery, • Efekt chwiania się - sinusoidalne ruchy kamery w poziomie i pionie • Zmiana FOV (Field of View) - oscylacja pola widzenia
18.	Niewyraźne widzenie odległych obiektów	Np. Rozmycie Gaussa dla pikseli o głębi większej niż dany próg d. W fotografii: Głębina ostrości - DOF (Depth of Field): $DOF \approx \frac{2u^2 Nc}{f^2}$ gdzie: • U - odległość od obiektu • N - wartość przysłony • C - maksymalny dopuszczalny rozmiar kręgu nieostrości • F - ogniskowa obiektywu $R = (D_i - D_o - Z_o) \cdot A_r$ gdzie: • D_i - odległość piksela od kamery • D_o - ustalona odległość ostrości • Z_o - zakres ostrości, w którym piksele pozostają ostre, • A_r - wielkość rozmycia Jeśli $D_i - D_o > Z_o$ i $D_i > D_o$, rozmycie obejmuje odległe obiekty. [Unity] DoF - Depth of Field - efekt głębi ostrości https://developer.nvidia.com/gpugems/gpugems/part-iv-image-processing/chapter-23-depth-field-survey-techniques

Lp.	Objaw	Wzór
		https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@7.1/manual/post-processing-depth-of-field.html
19.	Niewyraźne widzenie bliskich obiektów	$R = (D_i - D_o - Z_o) \cdot A_r$ gdzie: • D_i - odległość piksela od kamery • D_o - ustalona odległość ostrości • Z_o - zakres ostrości, w którym piksele pozostają ostre, • A_r - wielkość rozmycia Jeśli $D_i - D_o > Z_o$ oraz $D_i < D_o$, rozmycie obejmuje bliskie obiekty. [Unity] DoF - Depth of Field - efekt głębi ostrości (skonfigurowane odwrotnie niż dla odległych obiektów) Alternatywnie: Jeśli mamy obraz $I(x,y)$ i odległość od „bliska/ogniska” $d(x,y)$, to nowy obraz $I'(x,y)$ można zapisać jako: $I'(x,y) = \text{GaussianBlur}(I(x,y), \sigma(d(x,y)))$ Gdzie: • $\sigma(d) = k \cdot \max(0, D0 - d)$ • $D0$ - odległość graniczna, od której obiekt zaczyna być rozmyty (np. 50 cm w modelu VR). • k - współczynnik siły rozmycia. • $d(x,y)$ - lokalna odległość obiektu od obserwatora w obrazie (może być maską głębi lub ręcznie). Efekt: im bliżej, tym bardziej rozmyty obiekt. Unity: • Utworzenie zmiennej z odległością na której obraz ma być ostry • Odczytanie odległości każdego piksela za pomocą <i>depth texture</i> • Im bliżej niż wartość zmiennej tym silniejsze rozmycie
20.	Wrażenie rozmycia obrazu	Konwolucja obrazu z jądrem Gaussa: $G(u, v) = \iint I(x, y) K(u - x, v - y) dx dy$ gdzie: • G - obraz po rozmyciu,

Lp.	Objaw	Wzór
		• I – obraz przed rozmyciem, • K - kernel, jądro Gaussa Kernel – jądro Gaussa $K(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ gdzie: • σ – promień rozmycia Po dyskretyzacji: $g[x, y] = \sum_{i=-r}^r \sum_{j=-r}^r I[x-i, y-j] K[i, j]$ Jeżeli $I(x,y)$ to oryginalny obraz, to symulacja zmniejszonej ostrości i kontrastu może wyglądać tak: $I'(x,y) = (1 - \alpha) \cdot \text{GaussianBlur}(I(x,y), \sigma) + \alpha \cdot I(x,y)$ gdzie: • σ - promień rozmycia Gaussa (większe $\sigma \rightarrow$ większe rozmycie). • α - współczynnik mieszania, zmniejszający kontrast (0 - 1). Dodatkowo kontrast można skorygować: $I''(x,y) = 0.5 + \beta \cdot (I'(x,y) - 0.5), 0 < \beta < 1$ gdzie: • β - współczynnik zmniejszenia kontrastu (mniejsze wartości $\rightarrow$ mniejszy kontrast). Unity: • Zmniejszenie ostrości • Redukcja kontrastu • Redukcja nasycenia kolorów
21.	Zaburzenie rozpoznawania barw	Przykład macierzy dla deuteranopii: $M_{\text{deuteranopia}} = \begin{bmatrix} 0.625 & 0.375 & 0 \\ 0.7 & 0.3 & 0 \\ 0 & 0.3 & 0.7 \end{bmatrix}$ $R' = (0.625 \times R) + (0.375 \times G) + (0 \times B)$ $G' = (0.700 \times R) + (0.300 \times G) + (0 \times B)$

Lp.	Objaw	Wzór
		$B' = (0 \times R) + (0.300 \times G) + (0.700 \times B)$ • R, G, B to oryginalne wartości kolorów piksela • R', G', B' to nowe, zasymulowane wartości kolorów Unity: • Modyfikacja wartości RGB pikseli na ekranie • Protanopia → obniżenie wpływu czerwieni, przesunięcie percepcji w stronę zieleni • Deuteranopia → obniżenie wpływu zieleni • Tritanopia → obniżenie wpływu niebieskiego, przesunięcie w stronę koloru żółtego
22.	Problemy z widzeniem w słabym świetle	Jeżeli $I(x,y)$ to oryginalny obraz, a $L(x,y)$ - jego luminancja (jasność), to obraz symulujący hemeralopię można zapisać jako: $I'(x,y) = I(x,y) \cdot f(L(x,y))$ gdzie funkcja $f(L)$ obniża kontrast w ciemnych obszarach, np.: $f(L) = \frac{L^\gamma}{L^\gamma + (1-L)^\gamma}, \gamma > 1$ gdzie: • L - luminancja znormalizowana 0 - 1 • γ - współczynnik tłumienia ciemnych obszarów (np. 1.5 - 2) Unity: • Ograniczenie jasności (luminancji) obrazu • Redukcja kontrastu i nasycenia • Podbicie szumów • Utrata ostrości
23.	Niewyraźne widzenie w jasnym świetle	Jeżeli $I(x,y)$ to obraz, a $L(x,y)$ jego luminancja (jasność znormalizowana 0 - 1), to obraz symulujący efekt można zapisać jako: $\sigma(x,y) = k \cdot \max(0, L(x,y) - L0)$ $I'(x,y) = \text{GaussianBlur}(I(x,y), \sigma(x,y))$ gdzie: • $L0$ - progowa jasność, powyżej której zaczyna działać rozmycie. • k - współczynnik siły rozmycia dla jasnych obszarów. Unity: • Odczytywanie jasności każdego piksela • Jeśli jasność osiągnęła próg, następuje rozmycie obrazu (GaussianBlur)
24.	Zwiększona wrażliwość na światło i odbłaski	Jeżeli $I(x,y)$ to obraz, a $L(x,y)$ luminancja piksela (0 - 1), można wprowadzić efekt halo i nadwrażliwości: $I'(x,y) = I(x,y) + \alpha \cdot \text{GaussianBlur}(I(x,y), \sigma \cdot L(x,y))$ gdzie:

Lp.	Objaw	Wzór
		• α - siła nałożenia halo (np. 0.3-0.6). • σ - promień rozmycia zależny od jasności. Efekt: im jaśniejszy piksel, tym większe rozmycie wokół niego. Alternatywnie można dodać nieliniową kompresję kontrastu dla jasnych tonów: $I'' = I' \cdot (1 - \beta \cdot L(x,y)), \beta \in [0,1]$ gdzie: • β zmniejsza kontrast wokół bardzo jasnych punktów, tworząc efekt oślepienia. Unity: • Dla pikseli o wysokiej jasności zwiększenie intensywności + rozmycie
25.	Zaburzenia oceny odległości	Jeżeli mamy obraz $I(x,y)$ i mapę głębi $D(x,y)$ (odległość od obserwatora), można zasymulować zaburzenie oceny odległości poniższym wzorem: $\sigma(x,y) = k \cdot D(x,y) - D_{\text{śr}} $ $I'(x,y) = \text{GaussianBlur}(I(x,y), \sigma(x,y))$ gdzie: • $D_{\text{śr}}$ - średnia odległość (scena „płaska”). • k - współczynnik siły rozmycia. Efekt: obiekty bliższe lub dalsze niż średnia stają się lekko rozmyte, co utrudnia odczytanie ich rzeczywistej odległości. Unity: • Należy wykorzystać depth texture • Wprowadzić nieliniowe przekształcenie mapy głębi • Opcjonalnie można zastosować zmienne rozmycie

Źródła:

1. Lindeberg, T., "Discrete approximations of Gaussian smoothing and Gaussian derivatives," Journal of Mathematical Imaging and Vision, 66(5): 759–800, 2024.
2. Lowe, D.G. Distinctive Image Features from Scale-Invariant Keypoints. International Journal of Computer Vision 60, 91–110 (2004).
<https://doi.org/10.1023/B:VISI.0000029664.99615.94>
3. Gonzalez, R.C. and Woods, R.E. (2018) Digital Image Processing. 4th Edition, Pearson Education, New York
4. Physically Based Rendering: From Theory To Implementation, Matt Pharr, Wenzel Jakob, and Greg Humphreys
5. https://en.wikipedia.org/wiki/Gaussian_blur
6. https://en.wikipedia.org/wiki/Gaussian_function
7. https://en.wikipedia.org/wiki/Gaussian_filter
8. https://en.wikipedia.org/wiki/Brownian_motion
9. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0262457>
10. https://cave.cs.columbia.edu/old/publications/pdfs/Narasimhan_IJCV02.pdf

11. <https://www.cg.tuwien.ac.at/research/publications/2020/KROESL-2020-SVI/KROESL-2020-SVI-thesis.pdf>
12. https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
13. https://www.researchgate.net/publication/228951905_Visual_eye_disease_simulator
14. https://colour.readthedocs.io/en/v0.3.16_b/colour.blindness.html
15. https://docs.opencv.org/4.x/d3/dc1/tutorial_basic_linear_transform.html
16. <https://www.sharpcoderblog.com/blog/night-vision-image-effect-in-unity-3d>
17. [https://en.wikipedia.org/wiki/Bloom_\(shader_effect\)](https://en.wikipedia.org/wiki/Bloom_(shader_effect))
18. <https://developer.playcanvas.com/user-manual/graphics/posteffects/legacy/bloom/>
19. <https://developer.nvidia.com/gpugems/gpugems/part-iv-image-processing/chapter-23-depth-field-survey-techniques>
20. <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@7.1/manual/post-processing-depth-of-field.html>